Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САП



Методичні вказівки до лабораторної роботи № 3

з дисципліни: “Організація баз даних та знань”

на тему:

“**Написання SQL-сценаріїв для створення БД**”

Затверджено на засіданні кафедри
“Системи автоматизованого проєктування”
Протокол № 1 від 25.08.2026 р.

Львів-2026 р.

Написання SQL-сценаріїв для створення БД. Методичні вказівки до лабораторної роботи № 3 з курсу “Організація баз даних та знань” для студентів базового напрямку “Комп’ютерні науки” / Укл. О. М. Станкевич, О. І. Белей. – Львів: НУЛП, 2026. – 28 с.

Укладач: О. М. Станкевич
О. І. Белей

1. МЕТА РОБОТИ

Мета роботи – навчитися створювати базу даних в MySQL Workbench.

2. ТЕОРЕТИЧНІ ВІДОМОСТІ

2.1. Оператори для створення, наповнення та зміни структури таблиць бази даних (БД)

Для створення бази даних використовують команду **CREATE DATABASE (або SCHEMA).** Вона має такий синтаксис:

CREATE DATABASE [IF NOT EXISTS] ім'я_бази_даних;

Є декілька форм виконання цієї команди:

Перша форма.

CREATE DATABASE ім'я_бази_даних –

намагається створити базу даних, але якщо така база даних вже існує, то операція поверне помилку.

Друга форма.

CREATE DATABASE IF NOT EXISTS ім'я_бази_даних –

намагається створити базу даних, якщо на сервері відсутня БД з таким ім'ям.

Приклад. В MySQL Workbench CE (або в MySQL Command Line Client) виконаємо таку команду:

```
create database productdb;
```

або

```
create schema productdb;
```

Ці команди дають змогу створити на сервері БД з ім'ям productdb.

Ініціалізація бази даних

Після створення БД з нею можна здійснювати різні операції: створення таблиць, додавання і вибірка даних тощо. Але щоб можна було виконувати ці операції, потрібно вказати базу даних, яка буде використовуватися. Для цього застосовують оператор **USE**:

```
use productdb;
```

***Видалення бази даних* – DROP DATABASE | SCHEMA**

Для видалення бази даних застосовують команду DROP DATABASE, синтаксис якої:

DROP DATABASE [IF EXISTS] імя_бази_даних;

Команда має дві форми виконання.

Перша форма.

DROP DATABASE імя_бази_даних –

намагається видалити базу даних, але якщо така база даних відсутня на сервері, то операція поверне помилку.

Друга форма.

DROP DATABASE IF EXISTS імя_бази_даних –

намагається видалити базу даних, якщо на сервері є БД з таким ім'ям.

Приклад. Видалимо вище створену базу даних productsdb:

```
drop database productdb;
```

***Створення таблиць БД* – оператор CREATE TABLE.**

Цей оператор створює структуру таблиці, заповнити яку можна за допомогою оператора **INSERT**. Оператор **CREATE TABLE** визначає ім'я таблиці, а також її стовпці з типами та розмірами. Ім'я таблиці складається з двох частин: **назва БД.назва таблиці**. Таблиця повинна містити хоча б один стовпчик.

Приклад. Для створення таблиці student бази даних institute оператор матиме вигляд:

```
create table institute.student(
student_id int primary key auto_increment,
surname varchar(40),
name_ varchar(30),
birthday date,
stipend decimal,
kurs int,
group_ int);
```

***Типи даних* MySQL можна розбити на такі групи:**

- *числові типи (табл. 1);*
- *символьні типи (табл. 2);*
- *дати та часу (табл. 3);*
- *бінарні (табл. 3).*

Таблиця 1. Числові типи даних MySQL

Тип числових даних	Значення числових даних
TINYINT	цілі числа від -127 до 128, займає 1 байт
BOOL (BOOLEAN)	псевдонім для типу **TINYINT,** може зберігати два значення 0 і 1, може також приймати значення вбудо-ваних констант TRUE (1) і FALSE (0).
TINYINT UNSIGNED	цілі числа від 0 до 255, займає 1 байт
SMALLINT	цілі числа від -32768 до 32767, займає 2 байта
SMALLINT UNSIGNED	цілі числа від 0 до 65535, займає 2 байта
MEDIUMINT	цілі числа від -8388608 до 8388607, займає 3 байти
MEDIUMINT UNSIGNED	цілі числа від 0 до 16777215, займає 3 байти
INT	цілі числа від -2147483648 до 2147483647, займає 4 байта

INT UNSIGNED	цілі числа від 0 до 4294967295, займає 4 байта
BIGINT	цілі числа від -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807, займає 8 байт
BIGINT UNSIGNED	цілі числа від 0 до 18 446 744 073 709 551 615, займає 8 байт
DECIMAL (m, n) **псевдоніми NUMERIC, DEC, FIXED**	числа з фіксованою точністю; m – максимальна кількість цифр, які може зберігати число (від 1 до 65); n – максимальна кількість цифр у числі після коми (від 0 до m; за замовчуванням дорівнює 0)
FLOAT	дробові числа з плаваючою крапкою одинарної точності від -3.4028 * 1038 до 3.4028 * 1038, займає 4 байта; може приймати форму FLOAT (m, d), де m – загальна кількість цифр, а d – кількість цифр після коми
DOUBLE **псевдоніми** **REAL, DOUBLE PRECISION**	дробові числа з плаваючою крапкою подвійної точності від -1.7976 * 10308 до 1.7976 * 10308, займає 8 байт; може приймати форму DOUBLE (m, d), де m – загальна кількість цифр, а d – кількість цифр після коми

Таблиця 2. Символьні типи даних MySQL

Тип числових даних	Значення числових даних
CHAR(n)	стрічка фіксованої довжини цілі числа від -127 до 128, займає 1 байт
VARCHAR(n)	стрічка змінної довжини
TINYTEXT	текст довжиною до 255 байт
TEXT	текст довжиною до 65 Кбайт
MEDIUMTEXT	текст довжиною до 16 Мбайт
LARGETEXT	текст довжиною до 4 Гбайт

Таблиця 3. Типи даних MySQL для роботи з датою і часом

Тип числових даних	Значення числових даних
DATE	дата з 1 січня 1000 року до 31 грудня 9999 року (з “1000-01-01” до “9999-12-31”); за замовчуванням використовує формат yyyy-mm-dd; займає 3 байта
TIME	час від -838: 59: 59 до 838: 59: 59; за замовчуванням застосовує формат “hh: mm: ss”; займає 3 байта
DATETIME	об’єднує час і дату, діапазон дат і часу – з 1 січня 1000 року до 31 грудня 9999 року (з “1000-01-01 00:00:00” до “9999-12-31 23:59:59”); за замовчуванням використовує формат “yyyy-mm-dd hh: mm: ss”; займає 8 байт
TIMESTAMP	зберігає дату і час в діапазоні: від “1970-01-01 00:00:01” UTC до “2038-01-19 3:14:07” UTC; займає 4 байти
YEAR	зберігає рік у вигляді 4 цифр; діапазон доступних значень від 1901 до 2155; займає 1 байт.

Таблиця 4. Текстові та бінарні типи даних MySQL

Тип числових даних	Значення числових даних
текстові	
ENUM	одне значення зі списку допустимих значень; займає 1-2 байта
SET	кілька значень (до 64 значень) з деякого списку допустимих значень; займає 1-8 байт
бінарні	
TINYBLOB	бінарні дані у вигляді рядка довжиною до 255 байт
BLOB	бінарні дані у вигляді рядка довжиною до 65 КБ
MEDIUMBLOB	бінарні дані у вигляді рядка довжиною до 16 МБ
LARGEBLOB	бінарні дані у вигляді рядка довжиною до 4 ГБ

Можна дати декілька порад щодо вибору типів даних. Вони дають змогу уникнути повільного виконання запитів і потреби модифікації таблиць в майбутньому:

- Використовуйте якомога менший тип даних для полів таблиць. Наприклад, для зберігання чисел від 1 до 99 краще використати тип **TINYINT(2)** замість **INT**. Це впливає на швидкість пошуку і вибірки даних.
- Використовуйте рядки фіксованої довжини, якщо це можливо. Тобто, варто уникати типів **VARCHAR**, **TEXT** і **BLOB**. Це пришвидшить вибірку даних з середини рядків, оскільки ці дані будуть мати фіксовану адресу. Для цього всі поля таблиці повинні бути фіксованої довжини. За необхідності використання полів із типами **TEXT** або **BLOB**, їх можна виділити в окрему таблицю.

Атрибути стовпців і таблиць

PRIMARY KEY задає первинний ключ таблиці.

```
use productsdb;
create table customer (
    id int primary key,
    age int,
    first_name varchar (20),
    last_name varchar (20)
);
```

Первинний ключ унікально ідентифікує рядок у таблиці. Первинним ключем необов'язково повинні бути стовпці з типом int, може бути будь-який інший тип.

Встановлення первинного ключа на рівні таблиці:

```
use productsdb;
create table customers (
    id int,
    age int,
    first_name varchar (20),
    last_name varchar (20),
    primary key (id)
);
```

Первинний ключ може бути складеним. Такий ключ використовує відразу декілька стовпців, щоб унікально ідентифікувати рядок в таблиці. Наприклад:

```
create table order_lines (
    order_id int,
    product_id int,
    quantity int,
    price float,
    primary key (order_id, product_id)
);
```

Тут поля order_id і product_id разом утворюють складений первинний ключ. Тобто в таблиці order_lines не може бути двох рядків, де для обох із цих полів одночасно були б одні і ті ж значення.

AUTO_INCREMENT вказує, що значення стовпця буде автоматично збільшуватися під час додавання нового рядка. Цей атрибут працює для стовпців цілочисельного типу або з числами з плаваючою крапкою.

```
create table customers (
    id int primary key auto_increment,
    age int,
    first_name varchar (20),
    last_name varchar (20)
);
```

У цьому випадку значення стовпця id кожної нового доданого рядка буде збільшуватися на одиницю автоматично (системою).

UNIQUE вказує, що стовпець може зберігати тільки унікальні значення.

```
create table customers (
    id int primary key auto_increment,
    age int,
    first_name varchar (20),
    last_name varchar (20),
    phone varchar (13) unique
);
```

У цьому прикладі стовпець phone, який представляє телефон клієнта, може зберігати тільки унікальні значення. І ми не зможемо додати в таблицю два рядки, у яких значення для цього стовпця буде збігатися.

Також можна визначити цей атрибут на рівні таблиці:

```
create table customers (
    id int primary key auto_increment,
    age int,
    first_name varchar (20),
    last_name varchar (20),
    email varchar (30),
    phone varchar (20),
    unique (email, phone)
);
```

NULL і NOT NULL

Щоб вказати, чи може стовпець приймати значення **NULL**, при визначенні стовпця йому можна задати атрибут **NULL** або **NOT NULL**. Якщо цей атрибут явно не буде використаний, то за замовчуванням стовпець буде допускати значення **NULL**. Винятком є той випадок, коли стовпець виступає в ролі первинного ключа, тоді за замовчуванням стовпець має значення **NOT NULL**.

```
create table customers (
    id int primary key auto_increment,
    age int,
    first_name varchar (20) not null,
    last_name varchar (20) not null,
    email varchar (30) null,
    phone varchar (20) null
);
```

У цьому випадку стовпець age за замовчуванням буде мати атрибут **NULL**.

Зауваження!!!

Рекомендують, якщо можливо, завжди використовувати поля з обмеженням **NOT NULL**. Хоча це може збільшувати об’єм бази на диску.

Задання допустимих значень полів. Оператор CHECK дає змогу задавати умову, якій повинно задовольняти значення у цьому полі або сукупності полів.

Приклад.

```
create table institute.student (
student_id int primary key,
surname char(25) not null,
name char(10) not null,
stipend decimal check(stipend<900),
kurs int,
city char(15),
birthday date,
univ_id int
);
```

```
create table institute.student (
student_id int primary key,
surname char(25) not null,
name char(10) not null,
stipend decimal,
kurs int,
city char(15),
birthday date,
univ_id int unique,
check (stipend<900 and univ_id=10)
);
```

***Зміна структури таблиць БД* – оператор ALTER TABLE.**

Оператор **ALTER TABLE** використовують для додавання або зміни параметрів стовпців БД. Для додавання стовпців використовують умову **ADD COLUMN**, а для зміни – **CHANGE**. Під час додавання стовпця необхідно вказати назву таблиці, назву стовпця, його тип та довжину. В операторі також можна задати місце додавання стовпця. Якщо стовпець вставляємо на перше місце, то задається опція **FIRST**, якщо після деякого стовпця, то задається назва стовпця, після якого виконати вставку.

Якщо опцію розташування стовпця не вказано, то він розташовується на останньому місці.

Приклад.

```
alter table institute.student add column id2 int not null after student_id;
alter table institute.student add column id_final int (6);
alter table institute.student add column id1 int (8) first;
```

Зауваження!!!

Модифікація типу стовпця можлива лише тоді, якщо він порожній. Якщо стовпець заповнений, то його довжину не можна зменшувати. Під час зміни стовпця спочатку вказують його попередню назву, а потім – нову назву.

Приклад.

```
alter table institute.student change id2 id3 varchar(8) not null;
alter table institute.student change id3 id3 varchar(20);
```

Для видалення стовпця використовують умову **DROP**:

```
alter table institute.student drop column id2;
```

Додавання/скидання атрибутів та індексів.

За допомогою команди **ALTER TABLE** можна додати чотири різні типи атрибутів та індексів.

1) **ADD PRIMARY KEY** – ця команда додає індекс **PRIMARY KEY** до заданого стовпця (або стовпців).

Приклад.

```
alter table institute.student add primary key (student_id);
```

2) **ADD UNIQUE** – додає або створює унікальний індекс щодо заданих стовпців. Припустимо, гіпотетично ми просто хочемо унікальні імена в таблиці. До

Приклад.

```
alter table institute.student add unique (suname);
```

3) **ADD INDEX** – додає звичайний індекс до будь-якого стовпця.

Приклад.

```
alter table institute.student add index (kurs);
```

4) **ADD FULLTEXT** – це спеціальний тип покажчика, який використовують для пошуку тексту.

Приклад.

```
alter table institute.student add fulltext (sutname);
```

Видалення таблиць **– оператор DROP TABLE**.

Щоб видалити таблицю її попередньо потрібно очистити. Далі використовують оператор **DROP TABLE**.

Приклад.

```
drop table institute.student;
```

Додавання стрічок у таблицю – оператор INSERT INTO.

У реляційній СКБД можна визначити два варіанти оператора **INSERT**.

Однорядковий оператор **INSERT** дає змогу додавати в таблицю один новий рядок. Його широко використовують в повсякденних аплікаціях, наприклад, у програмах введення даних.

Багаторядковий оператор **INSERT** забезпечує вибірку даних з однієї частини бази даних, їх трансформацію і додавання в іншу частину. Використовують зазвичай під час пакетної обробки і створенні нових даних.

Однорядковий INSERT

insert into таблиця (стовп1, [стовп2, ...]) values (значення1, [значення2, ...])

Багаторядковий INSERT

insert into таблиця (стовп1, [стовп2, ...])
select (стовп1a, [стовп2a, ...])
from таблиця_вибірки where ...

До запису, що міститься в середині багаторядкового оператора **INSERT**, стандарт SQL вимагає деяких логічних обмежень:

- у запит не можна включати **ORDER BY**.
- таблиця результату запиту повинна містити таку ж кількість стовпчиків, що і оператор **INSERT**.
- запит не може бути запитом на об'єднання кількох різних операторів **SELECT**.
- ім'я цільової таблиці оператора **INSERT** не може бути присутнім у виразі **FROM** запиту на читання чи будь-якого запиту, вкладеного в нього. Отже, забороняється додавання таблиці саму в себе.

Запит виду:

```
insert into table_first (id, name, adress) values (NULL, 'Semen', 'Ostafova str.');
```

додасть до таблиці table_first новий рядок зі значеннями name_ = 'Semen', address = 'Ostafova str.', та дасть можливість системі автоматично вибрати значення для поля id.

А такий запит:

```
insert into table_second (id, city, phone)
select table_first.id, table_first.name_, table_first.adress
from table_first where address = 'Ostafova str.';
```

скопіює з першої таблиці поля table_first.name= table_second.city, table_first.adress= table_second. phone, table_first.id= table_second.id, тільки тих рядків де вказана адреса 'Ostafova str.'

2.2. Рекомендації, щодо використання назв БД та таблиць

1. **Нижній регістр.** Всі ідентифікатори повинні бути написані повністю в нижньому регістрі (малими літерами). Це, як правило, прискорює введення та запити БД.

first_name, але не **First_Name**

2. **Типи даних не є іменами.** Імена об’єктів БД, зокрема імена стовпців, повинні бути іменником, що описує поле або об’єкт. Уникайте використання слів, які є лише типами даних.
3. Використовуйте **зрозумілі назви таблиць та БД.**
4. **Символ підкреслення.** Якщо ім’я об’єкта складається з декількох слів, то їх потрібно розділяти символом підкреслення («_») замість пробілів. Це робить імена більш читабельними:

   word_count або **team_member_id**, аде не **wordcount** або **wordCount**
5. **Повні слова, а не абревіатури**. Імена об’єктів повинні бути повними англійськими словами. Взагалі уникайте скорочень, особливо таких, що без голосних.

   middle_name, але не **mid_nm**
6. Уникайте **зарезервованих слів**. Уникайте використання будь-якого слова, яке вважається зарезервованим словом у БД, яку ви використовуєте:

   name_, але не **name**
7. Назви БД та таблиць **повинні бути в однині, а не множині.**

   team, але не **teams**
8. **Уникайте використання чисел** у назвах БД та таблиць.

   test_table, але не **test_table_1**
9. **Поля первинних ключів одного стовпця повинні мати ім’я id**. Він короткий, простий і однозначний. Це означає, що коли ви пишете SQL, вам не потрібно запам’ятовувати імена полів, до яких ви можете приєднуватись за зовнішнім ключем.

```
create table selling(
id int primary key auto_increment,
created_at date,
date_placed date);
```

   Деякі довідники пропонують використовувати префікс назви таблиці в імені поля первинного ключа, тобто **person_id** замість **id**.
10. **Поля зовнішнього ключа повинні** бути комбінацією назви згаданої таблиці та назви полів.

2.3. Порядок виконання роботи

Для швидкої розробки БД будемо використовувати MySQL Workbench (**http://dev.mysql.com/downloads/installer**) – багатоплатформовий інструмент, що дає змогу легко адмініструвати БД, створювати нові БД, а також створювати запити на SQL. Так, на Windows після встановлення в меню Пуск ми можемо знайти значок програми і запустити її (рис. 1):

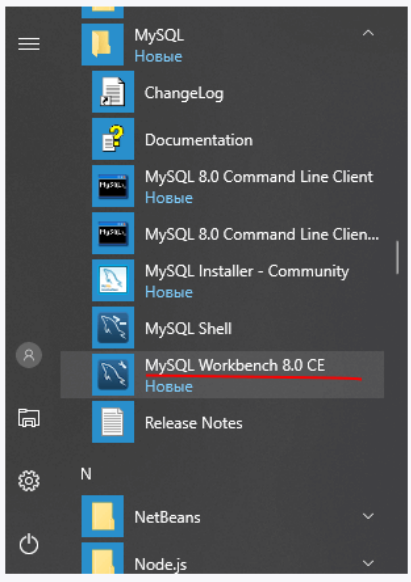


Рис. 1. Вибір MySQL Workbench у меню Пуск.

Відкриється вікно, у якому побачимо поле з назвою запущеного локально екземпляра MySQL (рис. 2):

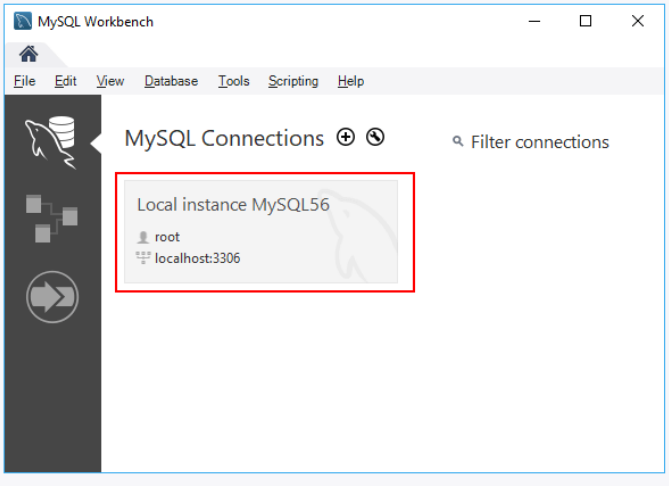


Рис. 2. Вікно із назвою запущеного локального екземпляра MySQL.

Клікнувши мишкою на ньому, відкриємо вікно для вводу пароля (рис. 3):

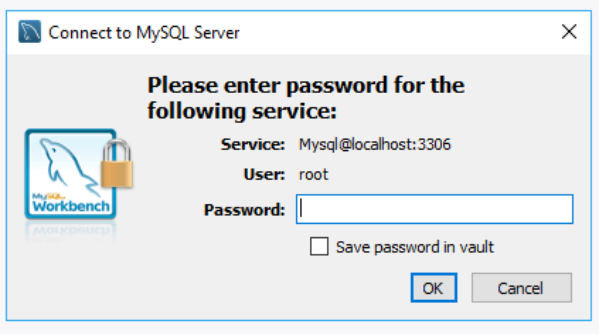


Рис. 3. Вікно для введення пароля.

Тут потрібно ввести пароль, який був встановлений для користувача root під час установки MySQL. Після успішного логіна відкриється вміст сервера (рис. 4):

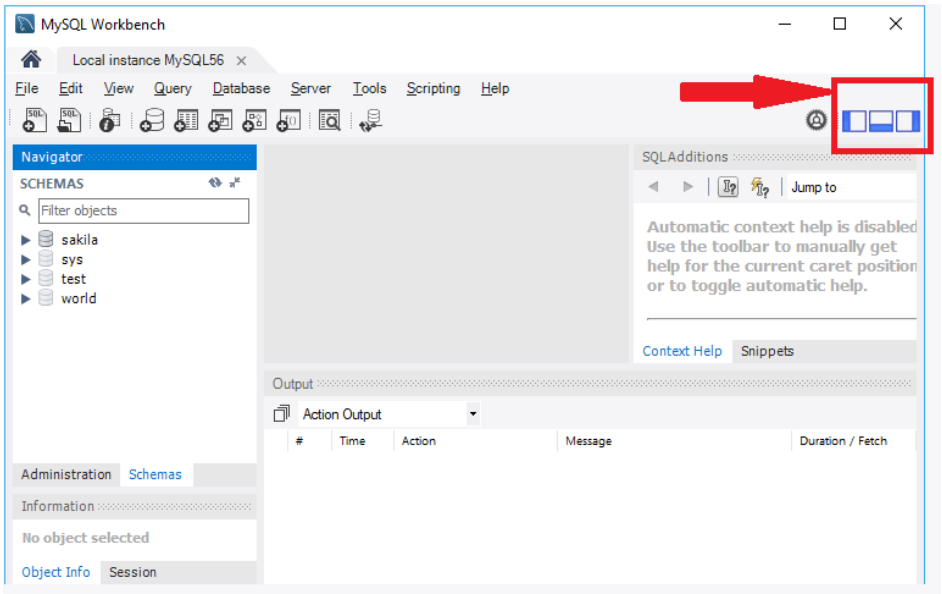


Рис. 4. Вікно сервера MySQL.

Зокрема, у лівій частині вікна SCHEMAS можна побачити перелік доступних бази даних.

Використовуючи піктограми “вікон” у правому верхньому куті (рис. 4), можна закривати або відкривати поля, які розташовані зліва, внизу і справа від центральної частини вікна програми.

Для створення бази даних *transport* спочатку натиснемо над списком баз даних на піктограму “SQL+” (рис. 5):

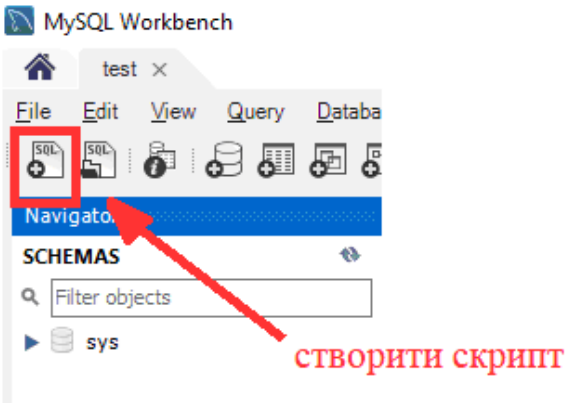


Рис. 5. Піктограма для створення скрипту.

Після цього в центральній частині програми відкриється вікно для введення скрипту SQL. Введемо оператори із рис. 6.

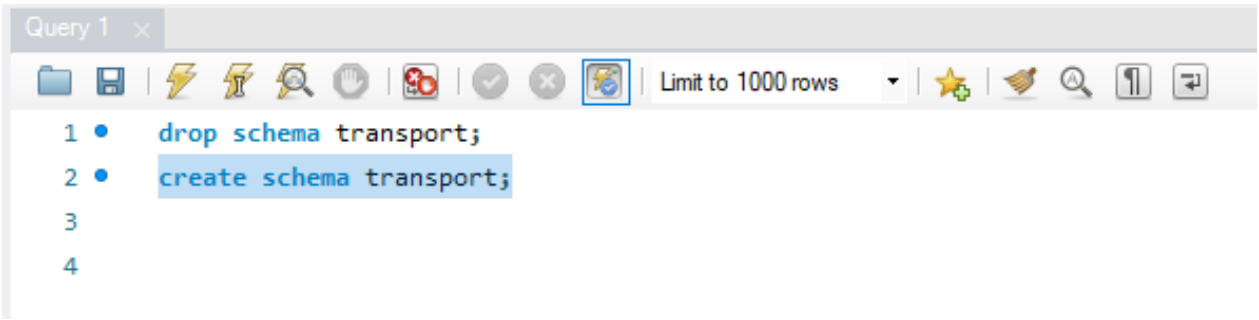


Рис. 6. Поле вводу скрипту SQL.

Оператор **drop schema** *transport* вводимо для спрощення подальшого редагування помилок під час створення бази даних. Якщо необхідно на деякий час зробити оператора неактивним, то перед ним прописуємо два мінуси “--” (коментар). Оператор **create schema** *transport* створює базу даних *transport*. На рис. 7 стрілками вказано основні елементи для створення та редагування скрипту SQL.

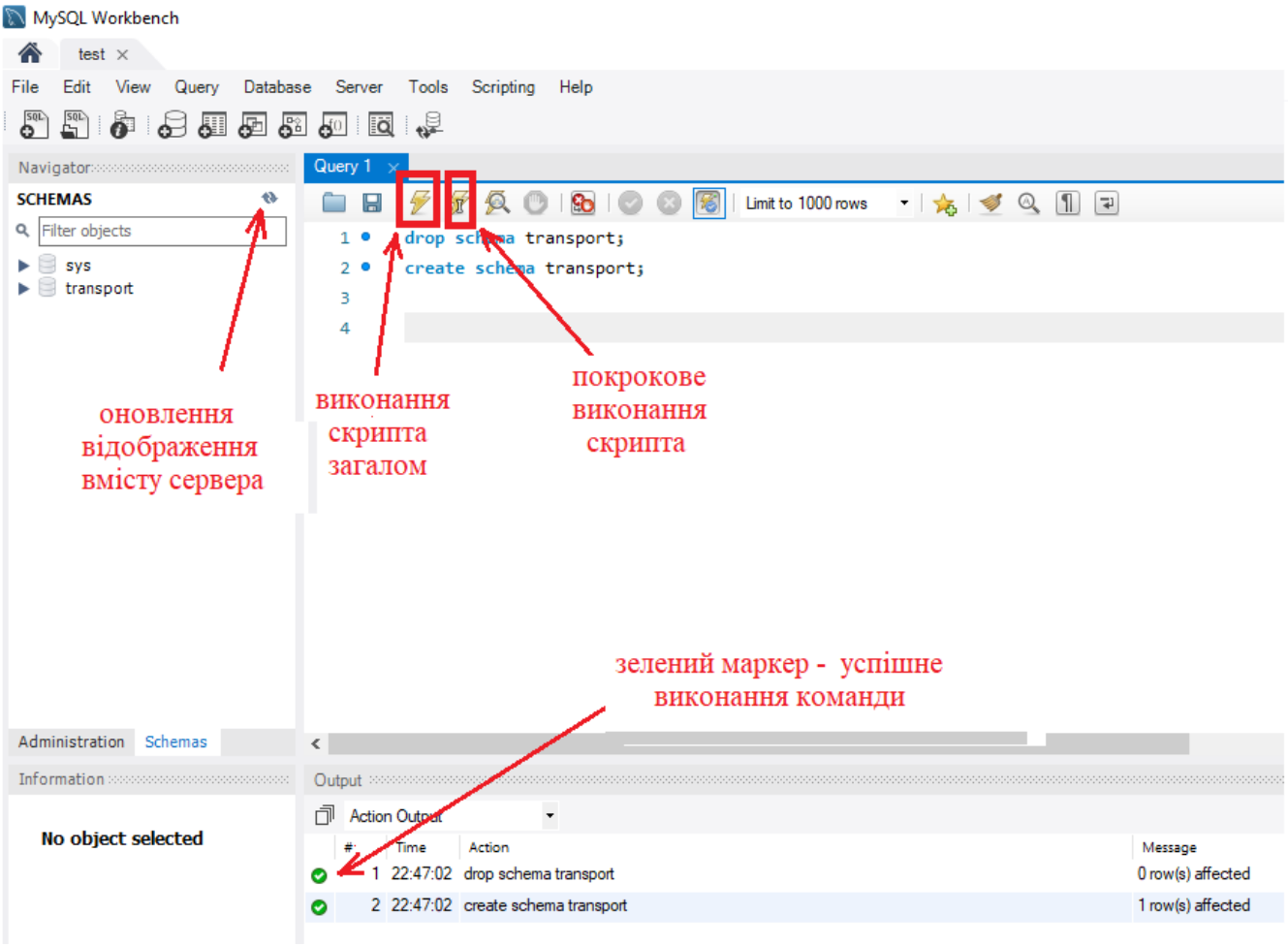


Рис. 7. Основні елементи вікна для створення та редагування скрипту SQL.

Для виконання скрипту в панелі інструментів використовуємо піктограму “блискавка”. Для покрокового виконання скрипту використовуємо піктограму “блискавка з курсором” (буде виконуватися або виділена частина скрипту, або рядок скрипту, на якому встановлений курсор).

Після виконання оператора чи набору операторів у нижній частині екрану відображається звіт про результат його виконання. Якщо з’явився маркер зеленого кольору, то скрипт виконано успішно, якщо ж червоного – у результаті виконання скрипту виявлено помилку.

Під час виконання операторів створення бази даних та таблиць необхідно оновлювати відображення вмісту сервера, натиснувши для цього піктограму “оновлення” у верхньому лівому кутку екрана, як показано на рис. 7.

Зазначимо, що зарезервовані слова SQL виділяються голубим кольором. Для того, щоб уникнути випадкового використання зарезервованого слова в назвах полів таблиць, рекомендують використовувати символ нижнього підкреслення “_”.

У результаті виконання введеного скрипту у списку об’єктів SCHEMAS з’явилася нова схема *transport*. У базі даних *transport* створимо декілька таблиць. Спочатку створюємо таблиці, які не мають зовнішніх ключів.

1. Таблиця Двигун (*autoengine*) (рис. 8).
   Поля таблиці, які ми визначимо:
   - Код двигуну(id_autoingine).
   - Назва (name_).
   - Потужність (power).
   - Об’єм (capacity).
   - Паливо (fuel).
2. Таблиця Виробник (*manufacturer*) (рис. 9).
   Поля таблиці, які ми визначимо:
   - Код виробника (id_manufacturer).
   - Назва виробника (name_).
   - Рік заснування (year_).
   - Країна (country).
3. Таблиця Автопарк (*autopark*) (рис. 10).
   Поля у цій таблиці задаємо такі:

- Унікальний код транспортного засобу (id), він буде типу integer – тобто цілий.
- Тип (type_) – легковий автомобіль, автобус, маршрутка. Задаємо його текстом.
- Номер (number_) – рядок.
- Рік випуску (year_) – можна задати або датою, або цілим числом.
- Код двигуна(id_engine) – знадобиться для іншої таблиці.
- Код виробника(id_manufacturer) – знадобиться для іншої таблиці.
- Модель (model).

```
16  create table transport.autoengine(
17  id int(15) primary key,
18  name_ varchar(25),
19  power int(15),
20  capacity int(15),
21  fuel varchar(8) not null
22  );
```

Рис. 8. SQL код для створення таблиці *autoengine*.

```
create table transport.manufacturer(
id int(15) primary key,
name_ varchar (25),
year_ smallint(4) not null,
country varchar(25) not null
);
```

Рис. 9. SQL код для створення таблиці *manufacturer*.

```
create table transport.autopark(
id int(15) primary key,
type_ varchar(25),
number_ varchar(15) not null,
year_ smallint(4) not null,
id_autoengine int(15) not null,
id_manufacturer int(15) not null,
model varchar(25) not null,
constraint autopark_autoengine_fk0 foreign key (id_autoengine) references transport.autoengine (id) on update no action  on delete no action,
constraint autopark_manufacturer_fk1 foreign key (id_manufacturer) references transport.manufacturer (id) on update no action  on delete no action
);
```

Рис. 10. SQL код для створення таблиці *autopark*.

Зауваження!!!

Інший спосіб створення зовнішніх ключів таблиць. Для коректної роботи системи з таблицями спочатку створюємо всі таблиці бази даних без визначення зовнішніх ключів. Після чого використовуємо синтаксис зміни зовнішнього ключа через зміну таблиці та задаємо послідовно всі зовнішні ключі всіх таблиць.

Якщо після команди створення бази даних задати команду **USE** ім'я_бази даних, то у назвах таблиць назву бази даних можна не вказувати.

Для бази даних *autopark* скрипт створення таблиць буде мати вигляд:

```
create schema transport;
use transport;

create table autopark(
id int(15) primary key,
type_ varchar(25),
number_ varchar(15) not null,
year_ smallint(4) not null,
id_autoengine int(15) not null,
id_manufacturer int(15) not null,
model varchar(25) not null);
```

```
create table autoengine(
id int(15) primary key,
name_ varchar(25),
power int(15),
capacity int(15),
fuel varchar(8) not null
);
create table manufacturer(
id int(15) primary key,
name_ varchar (25),
year_ smallint(4) not null,
country varchar(25) not null
);
alter table autopark
add
foreign key (id_autoengine) references autoengine (id) on update no action  on delete no action;
alter table autopark
add
foreign key (id_manufacturer) references manufacturer (id) on update no action  on delete no action;
```

Перейдемо до заповнення таблиць бази даних.
Для заповнення таблиці *manufacturer* введіть скрипт, зображений на рис. 11.

```
insert into transport.manufacturer(
id,
name_,
year_,
country)
values (
'1', 'Bugatti', '1909', 'France'),(
'2', 'AVTOZAZ', '1966', 'Russia'),(
'3', 'ZAZ', '1976', 'Ukraine'),(
'4', 'BMV', '1916', 'Germany'),(
'5', 'Ford', '1903', 'USA');
```

Рис. 11. Заповнення таблиці *manufacturer* (виробник) даними.

Щоб переглянути наповнення таблиці, потрібно ввести оператор

```
select * from transport.manufacturer;
```

або

```
select * from manufacturer;
```

та виконати його за допомогою піктограми “блискавка з курсором”, встановивши попередньо курсор на набраному операторі.

Результат виконання оператора зображено на рис. 12.

```
39 •  select * from transport.manufacturer;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

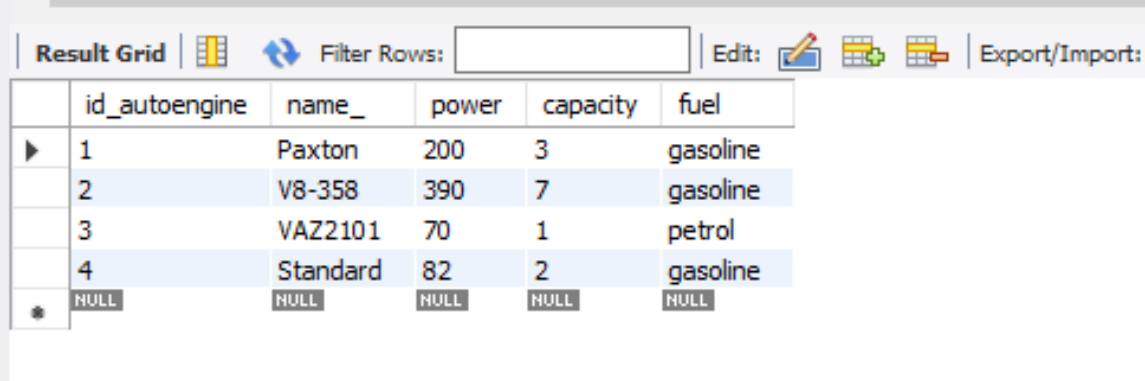
id_manufacturer	name_	year_	country
1	Bugatti	1909	France
2	AVTOZAZ	1966	Russia
3	ZAZ	1976	Ukraine
4	BMV	1916	Germany
5	Ford	1903	USA
NULL	NULL	NULL	NULL

Рис. 12. Перегляд таблиці *manufacturer* (виробник).

Аналогічно заповнюємо решту таблиць (рис. 13, рис. 15). На рис. 14 та 16 зображено результати виконання скриптів.

```
insert into transport.autoengine(
id,
name_,
power,
capacity,
fuel)
values(
'1', 'Paxton', '200', '3', 'gasoline'),(
'2', 'V8-358', '390', '7', 'gasoline'),(
'3', 'VAZ2101', '70', '1', 'petrol'),(
'4', 'Standard','82','2', 'gasoline');
```

Рис. 13. Заповнення таблиці *autoengine* (*двигун*) даними.



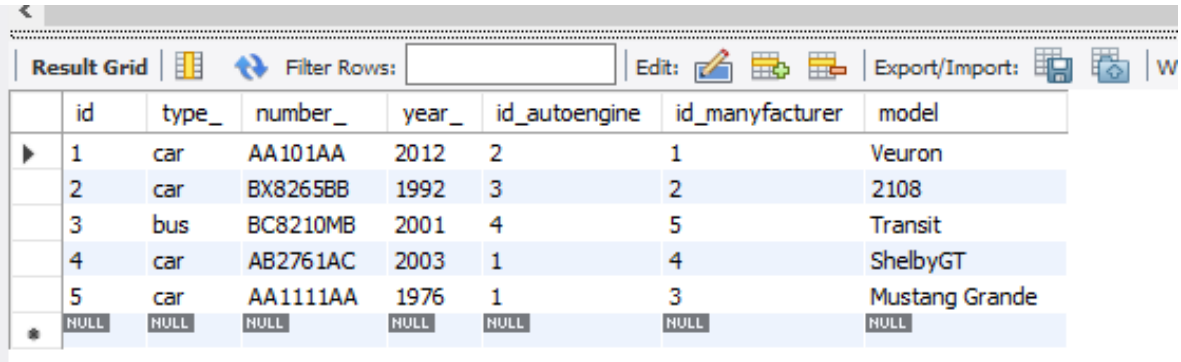
Result Grid | Filter Rows: | Edit: | Export/Import:

id_autoengine	name_	power	capacity	fuel
1	Paxton	200	3	gasoline
2	V8-358	390	7	gasoline
3	VAZ2101	70	1	petrol
4	Standard	82	2	gasoline
NULL	NULL	NULL	NULL	NULL

Рис. 14. Перегляд таблиці *autoengine* (*двигун*).

```
insert into transport.autopark(
id,
type_,
number_,
year_,
id_autoengine,
id_manyfacturer,
model)
values(
'1', 'car', 'AA101AA', '2012', '2', '1', 'Veuron'),(
'2', 'car', 'BX8265BB', '1992', '3', '2', '2108'),(
'3', 'bus', 'BC8210MB', '2001', '4', '5', 'Transit'),(
'4', 'car', 'AB2761AC', '2003', '1', '4', 'ShelbyGT'),(
'5', 'car', 'AA1111AA', '1976', '1', '3', 'Mustang Grande');
```

Рис. 15. Заповнення таблиці *autopark* (*автопарк*) даними



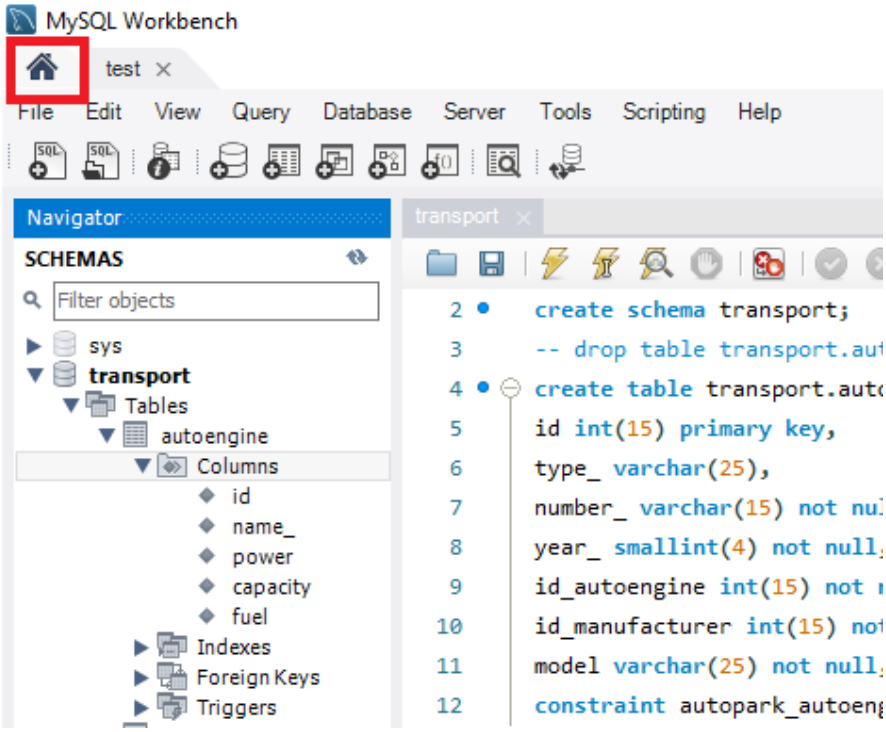
Result Grid | Filter Rows: | Edit: | Export/Import:

id	type_	number_	year_	id_autoengine	id_manyfacturer	model
1	car	AA101AA	2012	2	1	Veuron
2	car	BX8265BB	1992	3	2	2108
3	bus	BC8210MB	2001	4	5	Transit
4	car	AB2761AC	2003	1	4	ShelbyGT
5	car	AA1111AA	1976	1	3	Mustang Grande
NULL	NULL	NULL	NULL	NULL	NULL	NULL

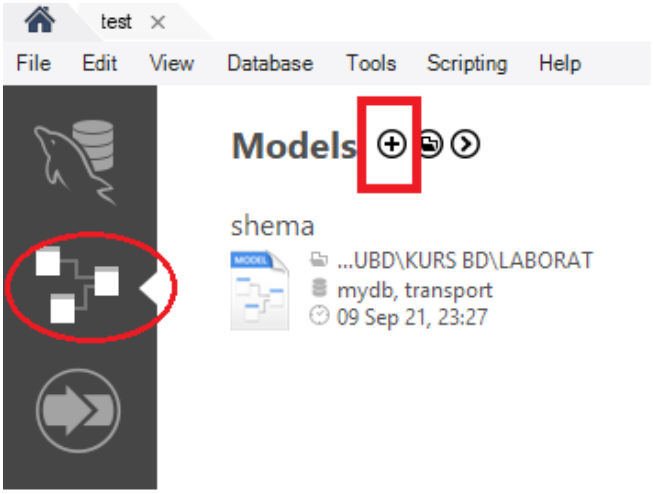
Рис. 16. Перегляд таблиці *autopark* (*автопарк*).

Згенеруємо EER-модель для побудованої БД. Для цього необхідно виконати такі кроки:

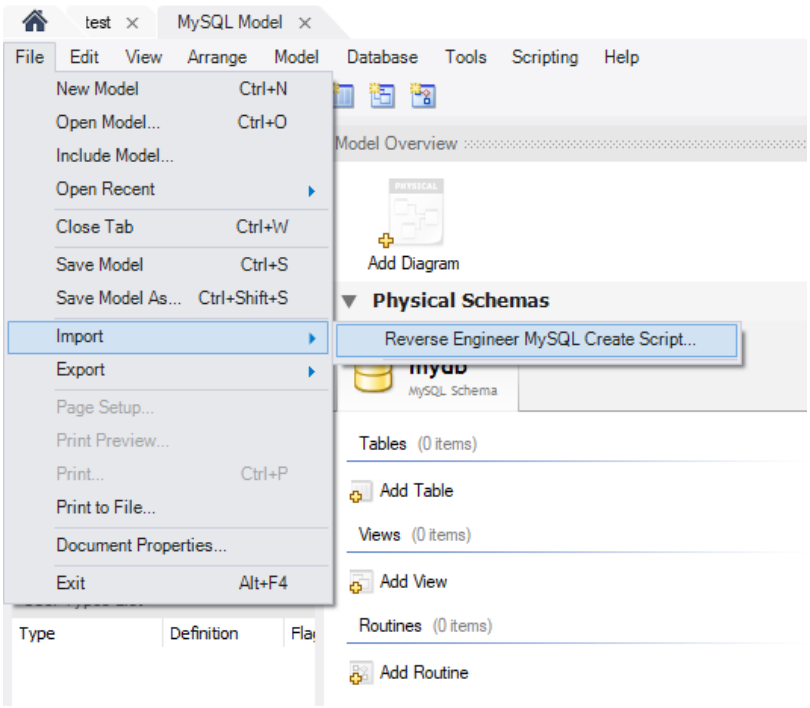
1. У лівому верхньому кутку вікна натиснути піктограму “хатка”.



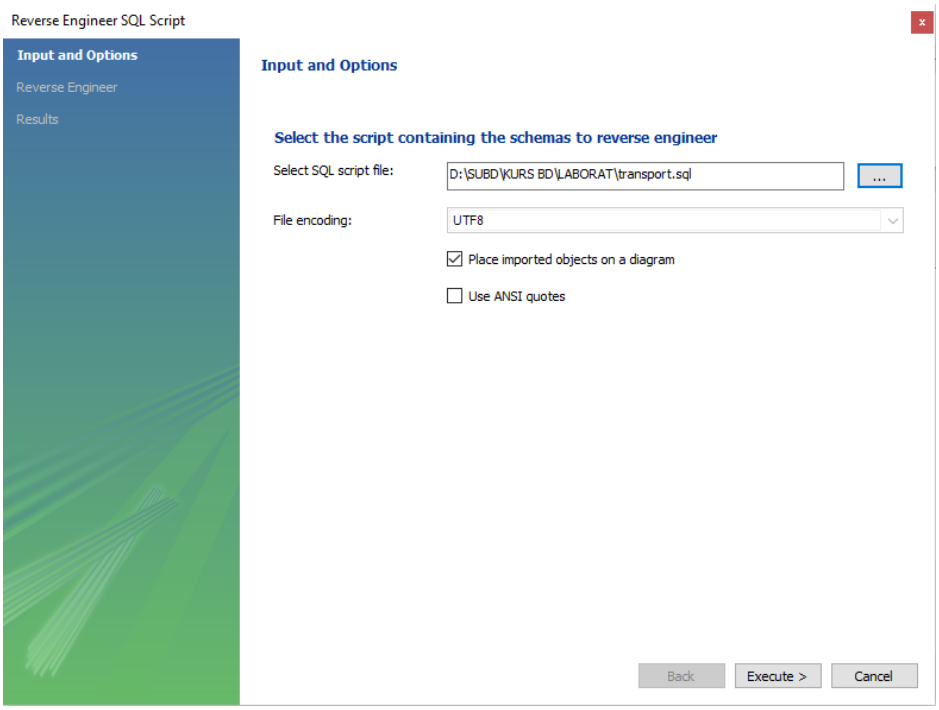
2. Перейти на зображення схеми та натиснути піктограму “плюс”.



3. У головному меню вибрати File → Import → Reverse Engineer MySQL Create Script…

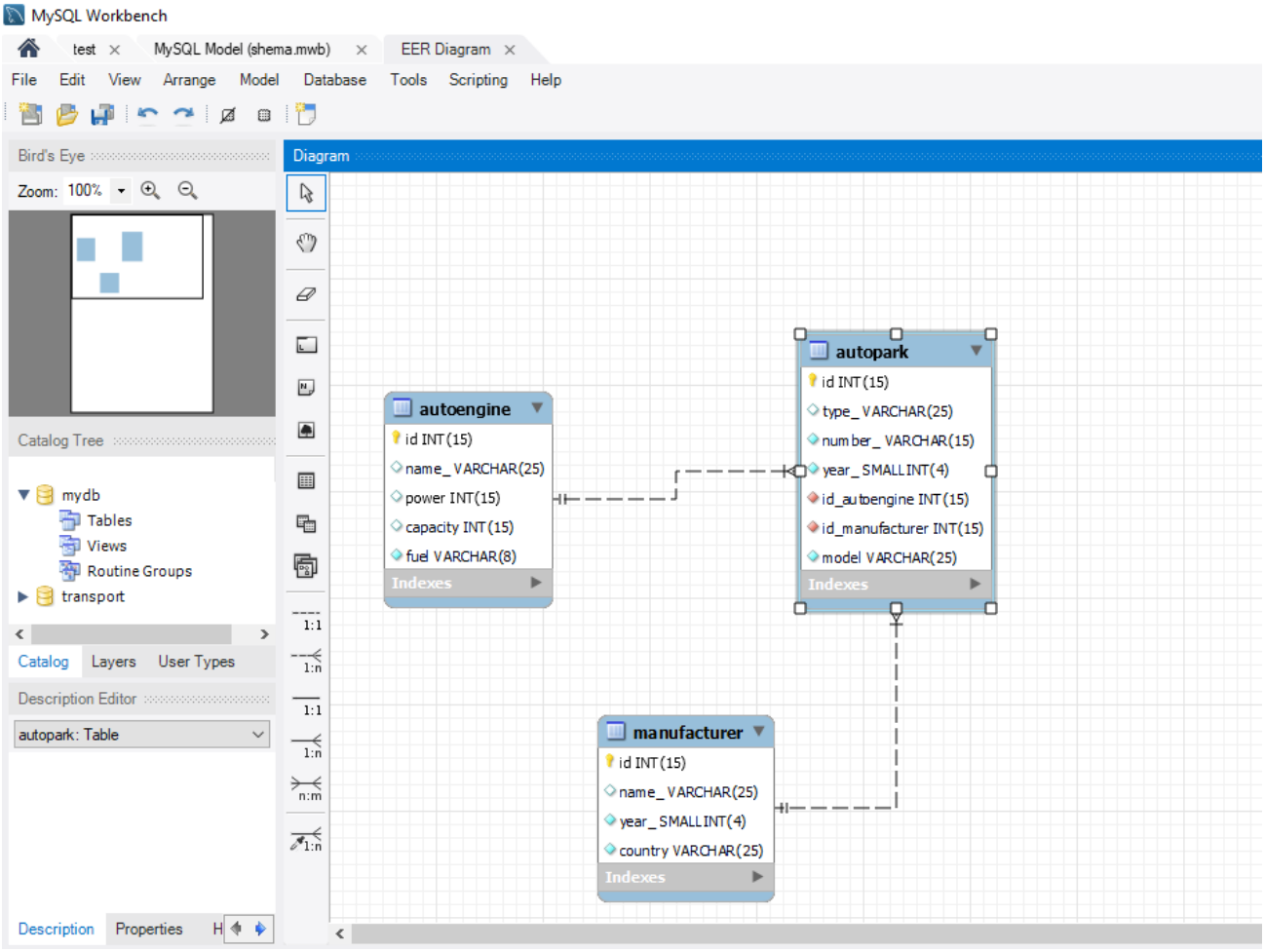


4. Вибрати файл зі скриптом та встановити прапорець у полі “Place imported odjects on a diagram”.



5. Після цього натиснути Execute → Next → Finish.

З’явиться вікно з EER-діаграмою, створеною за написаним скриптом.



3. Типові контрольні запитання

1. Як створити БД в My SQL?
2. За допомогою якого оператора створюють таблиці БД?
3. Які типи даних використовують в My SQL?
4. Які види ключів можна задати для стовпців таблиці?
5. Які зміни в структурі таблиці можна виконати за допомогою оператора ALTER TABLE?
6. Які типи індексів можна задати оператором ADD INDEX?
7. Як задати обмеження на дані в таблиці, видалити таблицю?
8. Види оператора INSERT?

4. Завдання для виконання лабораторної роботи

1. Створити БД для предметної області, вибраної у лабораторній роботі № 1.
2. Створити таблиці БД та визначити ключові поля в них.
3. Згенерувати EER-модель БД та перевірити її з побудованою раніше моделлю в середовищі проєктування DbDesigner.
4. Наповнити БД даними (***точність відношень не менше 10***).

Зміст звіту:
Титульний лист
Завдання 1. Скріншоти скриптів створення БД.
Завдання 2. Скріншоти скриптів створення таблиць БД.
Завдання 3. Скріншоти скриптів створення та заповнення даними таблиць БД.
Завдання 4. Скріншот схеми EER-модель БД для предметної області, вибраної у лабораторній роботі № 1.
Висновки.

5. Список літератури

1. MySQL Створення таблиці Підручник з прикладами. https://uk.myservername.com/mysql-create-table-tutorial-with-examples
2. Маніпулювання даними – оператори INSERT, UPDATE, DELETE. http://fcit.tneu.edu.ua/web-rozrobka/mysql/lektsiia-8-sqlmanipuliuvannia-danymy-operatory-insert-update-delete
3. Створення та модифікація таблиць та індексів – оператори CREATE, ALTER. http://fcit.tneu.edu.ua/web-rozrobka/mysql/581-lektsiia-10-sqlstvorennia-ta-modyfikatsiia-tablyts-ta-indeksiv-operatory-create-alter